Eng 3.7. Virtualization and Containers

Task 1 - Introduction

Cloud computing and virtualization have come to the forefront of technology as a solution for individuals, small and large businesses, and everything in between. Apart from cloud computing, virtualization can also be used to partition and create lab machines (VMs). This allows for atomic testing, development, research, and other uses that we will expand on throughout this room.

Task 2 - What is Virtualization

At its most basic level, virtualization is the concept of encapsulating the capabilities and features of a physical machine in a virtual environment, known as a **lab machine**. But why is virtualization needed? For most organizations and individuals, virtualization comes from a need of the following:

- **Decrease expenses:** Physical servers can be expensive, and virtualization can decrease the number of servers or other hardware, or even completely remove physical hardware from a company's infrastructure.
- **Scale:** Without properly implemented DevOps, it may be hard for a company to scale resources as server usage increases. Virtualization makes this process easier and can delegate a server's resources to lab machines as needed based on usage.
- **Efficiency:** Like scaling, virtualization can also make it easier to decrease the resources allocated to a lab machine if there is reduced usage.

Virtualization Technology: Virtualization abstracts or creates an **abstraction layer** over computer hardware. An abstraction layer allows a single device to be divided into multiple virtual computers, also known as lab machines (VMs). In simpler terms, this means that the lab machine will have access to *logical resources* that are abstracted away from the *physical resources*.

Virtualization is implemented using an engine-machine format, which means that a software or system creates an abstraction layer and allocates resources, while an operating system or application can then be installed on top of this virtualized environment. The operating system installed in a lab machine is known as a guest OS, as opposed to the host OS on which the virtualization engine is running. In the next task, we will introduce our first virtualization engine type - hypervisors.

Answer the questions below

> Is scalability a primary benefit of virtualization?

Y

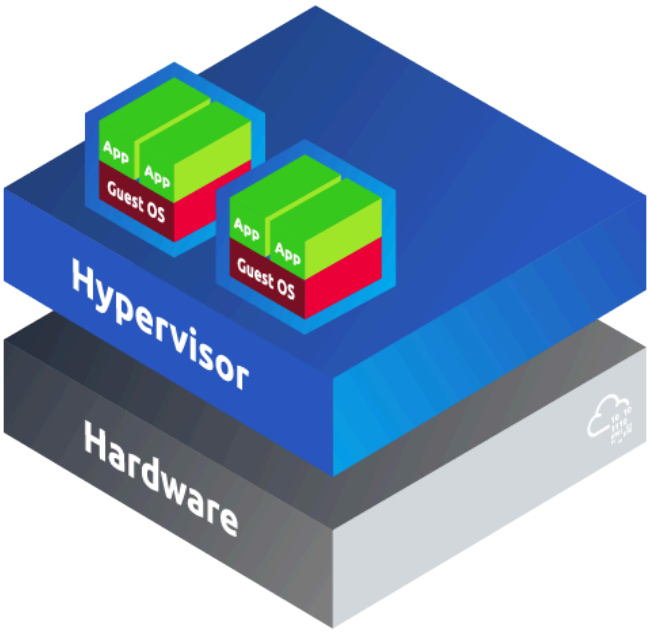
> What is the operating system of a lab machine often referred to as?

guest OS

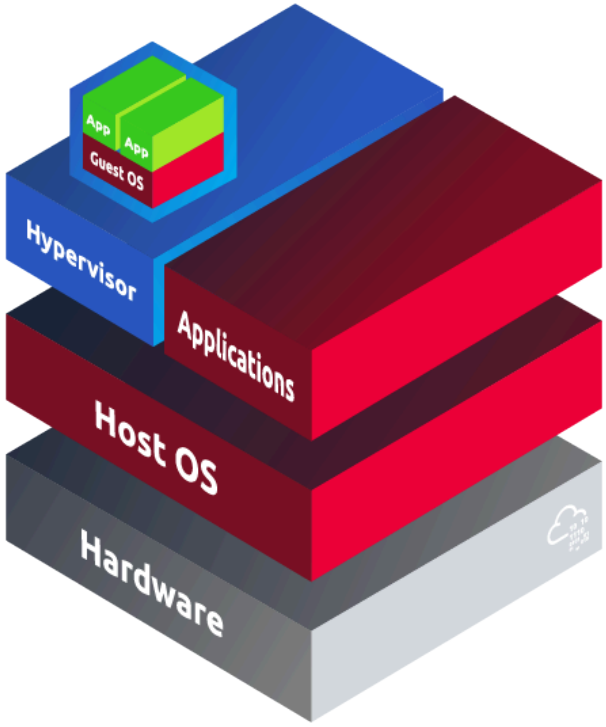
Task 3 - Hypervisors

A hypervisor provides the ability to create the abstraction layer between hardware and software. A hypervisor will also generally include some form of management application or software to provide an interface between the end user and the abstraction layer to create or load lab machines. Hypervisors are separated into two categories that are determined by their position relative to the hardware. They can either directly create a lightweight operating system on top of the hardware that is the hypervisor or add a hypervisor as an application on top of a pre-existing operating system.

Type 1 hypervisors, also known as **bare metal hypervisors**, create an abstraction layer directly between hardware and lab machines without a common operating system between them. Instead, the hypervisor is the operating system and is often *headless*, with only a web-based management portal remotely accessed. These hypervisors are designed for scale and to deploy a large number of lab machines at once. They are extremely lightweight to dedicate the most resources to lab machines. Below is a diagram of a type 1 hypervisor architecture. Examples of type 1 hypervisors include VMware ESXi, Proxmox, VMware vSphere, Xen, and KVM.



Type 2 hypervisors, also known as **hosted hypervisors**, create an abstraction layer from a software application built on top of a pre-existing operating system. Unlike type 1 hypervisors, type 2 hypervisors are often managed directly from the application through a GUI. These hypervisors are often designed for end-users or individuals such as developers and are not strictly designed to run a large number of lab machines for scale. Below is a diagram of a type 2 hypervisor architecture. Examples of type 2 hypervisors include VMware Workstation, VMware Fusion, VirtualBox, Parallels, and QEMU.



Answer the questions below

> What type of hypervisor is VirtualBox considered?

type 2

> What are type 1 hypervisors also known as?

bare metal hypervisors

Task 4 - Containers

Containers are the current solution to the issues encountered with hypervisors at scale. Containers have a lot in common with lab machines, but instead of being completely abstracted from the host operating system, containers share some properties with the host operating system. Containers have their own filesystem, a portion of computing resources (CPU, RAM), a process space, and more. Apart from the obvious benefits of being lightweight, containers are also *portable* and *robust* because they are not completely abstracted. Container engines are our second type of virtualization. As lab machines use a

hypervisor to create an abstraction layer for virtualization, containers use a container engine to create an abstraction layer using logical resources.

Answer the questions below

> Are containers completely abstracted from the host operating system?

N

Task 5 - Docker

Docker is a container platform and engine that is used to run Docker "images" as containers. Each Docker image is built of a base image, such as Alpine or Ubuntu, that is specifically built for use in containers and is lightweight. To build a Docker image, a Dockerfile must be created, which defines the base image for a container and any commands to be run.

Before starting the docker container, we must first introduce two new flags that must be used with the `run` command to allow the container to detach from the current terminal (`-d`) and expose ports externally (`-p`). Below is the required syntax to start the example Flask app in a Docker container, exposing the webserver to port 5000.

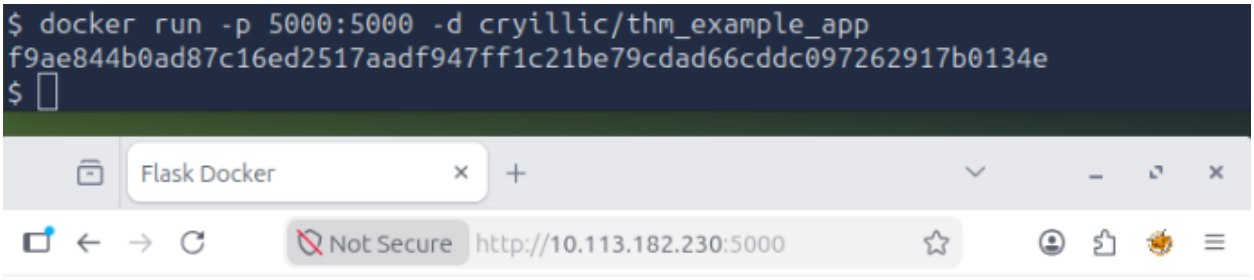
```
docker run -p 5000:5000 -d cryillic/thm_example_app
```

To verify that the web server is running in the Docker container, we can scan the container or access the designated port in a web browser. **Note:** Certain special characters may not be visible on the provided VM. When doing a copy-and-paste, it will still copy all characters

Answer the questions below

> What flag is obtained at MACHINE_IP:5000 (opens in new tab) after running the container?

THM{this_is_running_in_docker}



THM{this_is_running_in_docker}

Task 6 - Kubernetes

Kubernetes, also shortened to "**K8s**," is one such solution known as an **orchestration platform**. An orchestration platform aims to integrate into other products, such as Docker, and extend their capabilities or "synchronize" them with other products or applications. Kubernetes relies on these traditional virtualization models like hypervisors and containers and extends their uses, features, and capabilities. These capabilities and features include the following:

- **Horizontal scaling**: Unlike traditional "vertical" scaling, "horizontal" scaling refers to adding devices or machines to handle increased workload, rather than adding logic resources such as CPU or RAM.
- **Extensibility**: Clusters can be modified dynamically without affecting containers outside of the intended group.
- **Self-healing**: K8s can automatically restart, replace, reschedule, and kill containers that are not properly functioning based on user-defined health checks.
- **Automated rollouts and rollbacks**: K8s can progressively roll out changes to containers. As changes are made, it will monitor the application's health and decide whether to continue the rollout or rollback. This ensures the constant uptime of your cluster even if some containers fail.

Answer the questions below

Before proceeding, ensure all clusters are started by opening a terminal and running minikube start.

```
ubuntu@tryhackme:~$ minikube start
😄  minikube v1.32.0 on Ubuntu 20.04
✨  Using the docker driver based on existing profile
👍  Starting control plane node minikube in cluster minikube
🚜  Pulling base image ...
🔄  Restarting existing docker container for "minikube" ...
❗  This container is having trouble accessing https://registry.k8s.io
💡  To pull new external images, you may need to configure a proxy: https://mini
kube.sigs.k8s.io/docs/reference/networking/proxy/
🐳  Preparing Kubernetes v1.28.3 on Docker 24.0.7 ...-
```

> How many pods are running on the provided cluster?

1

```
ubuntu@tryhackme:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS        AGE
hello-tryhackme-66c4d4d69d-gb9nz    1/1     Running   1 (3m33s ago)   300d
```

> How many system pods are running on the provided cluster?

7

```
ubuntu@tryhackme:~$ kubectl get pods -A
NAMESPACE     NAME                                READY   STATUS    RESTARTS
   AGE
default       hello-tryhackme-66c4d4d69d-gb9nz    1/1     Running   1 (4m14s ago)
   300d
kube-system   coredns-5dd5756b68-z46xz            1/1     Running   1 (4m14s ago)
   300d
kube-system   etcd-minikube                       1/1     Running   1 (4m14s ago)
   300d
kube-system   kube-apiserver-minikube             1/1     Running   1 (4m14s ago)
   300d
kube-system   kube-controller-manager-minikube    1/1     Running   1 (4m14s ago)
   300d
kube-system   kube-proxy-4bkvs                    1/1     Running   1 (4m14s ago)
   300d
kube-system   kube-scheduler-minikube             1/1     Running   1 (4m14s ago)
   300d
kube-system   storage-provisioner                 1/1     Running   3 (2m38s ago)
   300d
```

> What is the pod name on the provided cluster?

hello-tryhackme-66c4d4d69d-gb9nz

```
ubuntu@tryhackme:~$ kubectl get pods
NAME                              READY   STATUS    RESTARTS        AGE
hello-tryhackme-66c4d4d69d-gb9nz  1/1     Running   1 (3m33s ago)   300d
```

> What is the deployment name on the provided cluster?

hello-tryhackme

```
ubuntu@tryhackme:~$ kubectl get deployments
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
hello-tryhackme   1/1     1            1           300d
```

> What port is exposed by the service in question 5?

443

```
ubuntu@tryhackme:~$  kubectl get service
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   300d
```

> How many replica sets are deployed on the provided cluster?

1

```
ubuntu@tryhackme:~$ kubectl get rs
NAME                        DESIRED   CURRENT   READY   AGE
hello-tryhackme-66c4d4d69d  1         1         1       300d
hello-tryhackme-b6545c585   0         0         0       300d
```

> What is the replica set name on the provided cluster?

hello-tryhackme-66c4d4d69d

```
ubuntu@tryhackme:~$ kubectl get rs
NAME                        DESIRED   CURRENT   READY   AGE
hello-tryhackme-66c4d4d69d  1         1         1       300d
hello-tryhackme-b6545c585   0         0         0       300d
```

> What command would be used to delete the deployment from question 5?

kubectl delete deployment hello-tryhackme